Chapter 4

Implementation and Results

This chapter covers the physics-informed neural network (PINN) we built for the Lorenz-1960 system, the accuracy it reached, and only those parts of the code that carry the method. Five Python modules make up the implementation, all of them built on PyTorch. `config.py` holds the settings, `pinn.py` the network and loss, `train.py` the training loop, `history.py` the training record, and `figures.py` the plots. Rather than retype the reference solver and the error metrics, we import both from the baseline module of Chapter 3. The reference trajectory is used for one thing only, to score the network after training has finished. It plays no part in the loss.

4.1 Network and Loss Function

Four hidden layers of 60 tanh units map the time t to the state (x, y, z). The weights use Xavier initialisation and the biases start at zero.

The initial condition is not left for the network to learn. We build it into the architecture through a trial solution,

$$u_T(t) = u_0 + g(t)\, N(t), \qquad g(t) = \frac{t - t_0}{t_f - t_0}, \tag{4.1}$$

where $N(t)$ is the raw network output. Since the factor g vanishes at $t = t_0$, the equality $u_T(t_0) = u_0$ holds exactly. Three lines of code carry the whole construction.

```
def forward(self, t):                    # trial solution
    g = (t - self.t0) / (self.tf - self.t0)
    return self.u0 + g * self.net(t)      # g(t0) = 0, so the IC is exact
```

The construction follows Lagaris et al. [10] and Babni et al. [6]. With the initial condition already satisfied, the loss can drop both the penalty term that would otherwise enforce it and the weight needed to balance that term against the residual. Chapter 3 listed this kind of loss balancing among the reasons to avoid PINNs; the trial solution disposes of it. We also implement a soft-penalty variant in the style of Raissi et al. [13], selected by `Config(ic="soft")`. Keeping both available leaves room for a comparison of the two treatments later.

To measure how badly a candidate solution breaks the equations we use the residual $r(t) = du_T/dt - f(u_T)$, in which f is the right-hand side of Chapter 2. Automatic differentiation supplies du_T/dt exactly, free of the truncation error that a finite-difference estimate would introduce. Setting `create_graph=True` keeps that derivative itself differentiable, as the optimiser requires.

```
def residual(model, t):                  # r(t) = du/dt - f(u)
    u = model(t)
    d = [grad(u[:, j].sum(), t, create_graph=True)[0][:, 0] for j in range(3)]
    return ode_residual(u, torch.stack(d, dim=1), model.coeffs)

def ode_residual(u, dudt, c):            # the governing equations enter here
    x, y, z = u[:, 0], u[:, 1], u[:, 2]
    f = torch.stack([c[0]*y*z, c[1]*x*z, c[2]*x*y], dim=1)
    return dudt - f
```

Here `grad` is `torch.autograd.grad`. The function `ode_residual` is the single point at which the governing equations enter the program. Its `c` argument comes from `lorenz1960_coefficients(k, l)` in the baseline module. At $k = 2$ and $l = 1$ that call returns the values printed in Chapter 2, namely $(-0.10, 1.60, -0.75)$.

Averaging the squared residual over the collocation points $t_1, \ldots, t_N$ gives the loss, $\mathcal{L}(\theta) = \frac{1}{N}\sum_{i=1}^{N} \|r(t_i)\|^2$. Nothing in that expression refers to the true solution. The network therefore trains without ever seeing it.

4.2 Training Protocol

A Latin hypercube sample supplies the 3000 collocation points. It spreads them more evenly across the interval than uniform random draws.

```
t0, tf = cfg.t_span
sample = qmc.LatinHypercube(d=1, seed=cfg.seed).random(cfg.n_collocation)
t = t0 + (tf - t0) * sample              # map the unit sample onto [t0, tf]
```

Training runs full batch. Every step therefore uses all 3000 points. Table 4.1 lists the settings, which follow the forward-PINN protocol of Matthews and Bihlo [4], Section 4.1. An L-BFGS polishing stage is implemented but switched off for this run. What we report below is therefore Adam only.

Table 4.1: PINN Training Configuration

Setting	**Value**	**Setting**	**Value**
Layers × width	4×60	Optimiser	Adam, full batch
Activation	tanh	Iterations	20,000
Collocation points	3000, Latin hypercube	Learning rate	10^{-3} to 10^{-4}, linear
Initial condition	hard, Equation 4.1	Random seed	0

4.3 Results

Figure 4.1 shows the solution, the signed error, and the training loss. Table 4.2 reports the error against the DOP853 reference over 1001 points. Its combined row gives the Euclidean magnitude of the error vector rather than an average over the three states.

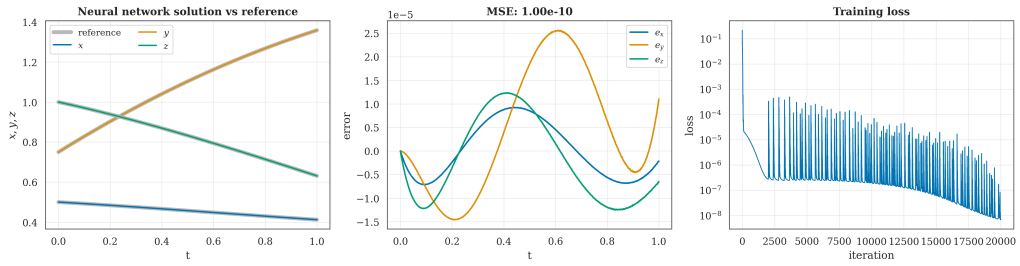


Figure 4.1: PINN solution, signed error, and training loss over 20,000 iterations.

Table 4.2: PINN error against the DOP853 reference.

State	**MAE**	**RMSE**	**Max. abs. error**
x	5.20×10^{-6}	5.77×10^{-6}	9.24×10^{-6}
y	1.12×10^{-5}	1.36×10^{-5}	2.56×10^{-5}
z	8.27×10^{-6}	9.04×10^{-6}	1.25×10^{-5}
Combined L_2	1.66×10^{-5}	1.73×10^{-5}	2.60×10^{-5}

Over the run the training loss falls from 2.17×10^{-1} to 6.82×10^{-9}. Against states that span roughly 0.41 to 1.36, an error near 10^{-5} amounts to about five correct decimal places, obtained from the equations alone. Nine automated checks cover the exact initial condition, the residual definition, loss reduction in both IC modes, and figure generation; all nine pass, and the run reproduces from seed 0.

Two limitations qualify the table. The errors are taken from the final iteration, and the error against the reference has not settled by that point: across the last 5000 iterations it swings through nearly four orders of magnitude, with the reported value sitting near the bottom of that range. Training also runs in single precision, and that may be what sets the error floor. A repeat with several seeds, best-checkpoint selection, and double precision is under way.

Running `python run_pinn.py` repeats the whole pipeline. Since `history.py` stores the per-iteration telemetry beside the checkpoint, `rebuild_figures()` can redraw every figure from a finished run, working from the stored record instead of a fresh training pass.